

pif_eth — Project Report

8b/10b line-coded Ethernet transmitter over the PRU Peripheral Interface, generated on 2026-07-20 in the `pru_simulator` project.

1. Initial prompt

The project was requested with the following prompt (verbatim):

generate a project folder under source folder called pif_eth. This folder hosts a project which transfers Ethernet frame which is 8/10B line coded over 3 peripheral interface using 125 Mbit line rate - 100 Mbit raw Ethernet. The 8B/10B line code is a single LUT using LBCO from ICSS DRAM0. Use PRU0 single core implementation. Example Ethernet frame has two options. Option 1 using pseudo random number generate from Byte 1 to byte 128 + 4 bytes of CRC32. The frame is meant for bit error rate testing and has no pre-amble and no Ethernet header. All payload bytes are random number. Second frame is a standard UDP packet with Ethernet header UDP header, UDP payload "Hello World Text" and CRC32 which is Ethernet frame check sum. The CRC32 in a second stage will use broadside widget CRC32. In first implementation can be firmware generated. Data flow option 1: pseudo random number generator with defined seed -> pack into 128 byte Ethernet Frame. Calculate CRC32 in firmware which is bytes 125-128. The 128 bytes reside in memory. Send of 8/10B line coded frame is in realtime. Load 32 bit from memory, LUT to get 40 bit, send 32 bits to TX fifo of peripheral interface. Wait for Fifo half empty, load next bytes from DRAM, LUT and compile next 16 bits which is 8 bits remaining from previous and 8 bit from new LUT, continue like that till end of frame. Test the code with MCP server for PRU. Ideally generate a wireshark trace for the frames generated. Keep a frame counter and run the test like 100 frames.

Four points in the prompt were ambiguous and were clarified before implementation (see §2).

2. Design choices and decisions

2.1 Clarified with the requester

Question	Decision	Rationale
"over 3 peripheral interface" — one channel or striped across three?	Single channel (ch0) , one continuous bit stream	Matches the byte-repacking data flow in the prompt; striping deferred
"single LUT" vs. true 8b/10b (disparity-dependent, two codes per byte)?	True 8b/10b with running disparity , single LUT holding both RD variants	Requester wanted realistic line code; still a single DRAM0 table
BERT frame length — 128 or 132 octets?	132 octets (128 payload + 4 CRC)	Requester's choice

Wireshark trace scope	Both frame types to pcap	BERT is malformed Ethernet (expected); UDP dissects cleanly
-----------------------	---------------------------------	---

2.2 Line code

- IBM/ANSI **8b/10b** (Widmer & Franaszek): 5b/6b on the low 5 bits + 3b/4b on the high 3 bits, transmitted `a b c d e i f g h j` (a first on the wire), with the D.x.A7 alternate to break runs of five.
- The reference encoder (`codec.py`) is validated by **invariants** rather than by transcribing the ISO tables: running disparity stays ± 1 at every symbol boundary, per-symbol disparity is $-2/0/+2$, no run exceeds 5 bits, and decode is a bijection over all 256 octets in both disparity contexts. The firmware and the host decoder only need to agree with this reference (hardware interop is out of scope for a simulator BER test).
- **K28.5** commas are emitted as idle fill / inter-frame delimiters, which also give the decoder symbol alignment.

2.3 Line rate

The bit clock is `source / ((frac+1) * (div+1))`; exact 125 MHz is not integer-divisible from the 200 MHz core clock. The model uses `TXCFG div = 7` → **25 MHz**, which is representable and leaves the single core ample headroom to keep the 4-deep TX FIFO fed. Because the encoder and the host decoder share the configured period, the absolute rate does **not** affect the byte stream or the measured BER; **125 Mbaud** remains the documented physical target.

2.4 Frame structure

- **BERT (option 1)** — no preamble, no header. 128 octets from a firmware **xorshift32** PRNG (defined seed, state persists across frames) + 4-octet **CRC-32** = 132 octets.
- **UDP (option 2)** — Ethernet/IPV4/UDP, payload `"Hello World Text"`, padded to the 60-octet Ethernet minimum + 4-octet FCS = 64 octets.
- **CRC-32** is firmware-generated (bit-serial, reflected `0xEDB88320`, init/final `0xFFFFFFFF`), identical to `zlib.crc32`. A broadside CRC accelerator is noted as the second stage.

2.5 Firmware architecture

- Single-core **PRU0**, channel 0, **continuous** TX mode.
- **Self-configuring**: writes `GPCFG0/TXCFG/CH0CFG0` itself; no host register setup. Channel 0 is selected with a byte-2 R30 write — a byte-0 write would strobe the TX FIFO and inject a stray `0x00` byte (this bug was found and fixed during bring-up, see §5).
- Each Ethernet frame is a **separate continuous-mode burst**: payload/CRC are generated *before* `tx_go`, so the FIFO never under-runs during generation (an unbroken multi-frame stream would starve while the PRNG/CRC loops run).
- Subroutines with **fixed return-address registers per nesting level** (`r29` → `r28` → `r26` → `r27`) stand in for a call stack the ISA lacks.
- A per-frame **go-flag handshake** (`0x0418`) lets the host release and capture one frame at a time.

2.6 Host verification / capture

The perif TX timeline advances as PRU0 steps, recording timestamped serial-line transitions. The driver releases one frame, steps until the frame counter increments, then reconstructs that burst's bits by sampling the recorded line at bit centres (`go + (k+½) * period`), 8b/10b-decodes them, checks

them against the golden reference (**BER = 0**), and writes the decoded frames (FCS stripped) to a classic-format pcap (Ethernet link type). The firmware ends in a spin (not `halt`) so the perif keeps advancing while the last burst drains.

3. Files generated

All under `source/pif_eth/` unless noted ($\approx 1,400$ lines total).

File	Lines	Purpose
<code>pif_eth_tx.asm</code>	260	PRU0 firmware: self-config, PRNG, CRC-32, 8b/10b encode, stream
<code>codec.py</code>	155	8b/10b encode/decode + DRAM0 LUT builder (golden reference)
<code>crc32.py</code>	37	Ethernet FCS (bit-serial + <code>zlib</code> references)
<code>prng.py</code>	30	xorshift32 PRNG reference
<code>frames.py</code>	83	BERT and UDP frame builders
<code>decoder.py</code>	63	bit/symbol stream $\rightarrow$ frames (comma-delimited)
<code>pcap.py</code>	30	minimal classic-pcap writer (Ethernet link type)
<code>driver.py</code>	191	run firmware on the sim, decode, check BER, write pcap
<code>seed_ui.py</code>	74	seed the running UI simulator's DRAM0 over WebSocket
<code>__init__.py</code>	5	package docstring
<code>README.md</code>	96	package overview / quickstart
<code>PROJECT_REPORT.md</code>	—	this document
<code>tests/test_pif_eth.py</code>	222	18 tests (pure-Python + firmware + MCP)
<code>docs/superpowers/specs/2026-07-20-pif-eth-8b10b-tx-design.md</code>	94	design spec
<code>docs/superpowers/plans/2026-07-20-pif-eth-8b10b-tx.md</code>	56	implementation plan / as-built

Also modified: `.gitignore` (ignores the generated `source/pif_eth/traces/` pcap artifacts).

DRAM0 memory map

Address	Contents
0x0000	8b/10b encode LUT, $256 \times u32$
0x0400	<code>num_frames</code> (u32)
0x0404	<code>mode</code> (u32): 0 = PRNG payload, 1 = preloaded payload
0x0408	<code>prng_state</code> / <code>seed</code> (u32)

0x040C	payload_len (u32)
0x0410	frame_counter (u32, published)
0x0414	burst_pushed bytes (u32, published)
0x0418	go flag (u32, host→firmware handshake)
0x0500	frame core buffer (payload + 4-byte FCS)

4. Testing

The strategy is **golden-reference first, then firmware**: a pure-Python reference layer defines the exact byte-level behaviour the firmware must reproduce, so firmware bugs surface as byte mismatches against a known-good model. `tests/test_pif_eth.py` has **18 tests** (all green; full repo suite 1159 passed):

Pure-Python reference (no simulator)

- `test_encode_decode_roundtrip_all_bytes_both_rd` — bijection over all 256 octets, both disparities.
- `test_running_disparity_stays_bounded` / `test_no_run_longer_than_five` — 8b/10b legality invariants.
- `test_comma_flips_disparity`, `test_dram0_lut_matches_encoder`, `test_stream_decode_with_commas`.
- `test_crc32_bitwise_matches_zlib`, `test_crc32_check_value` ("123456789" → 0xCBF43926), `test_prng_*`, `test_bert_frame_shape`, `test_udp_frame_valid_length_and_fcs`, `test_full_frame_line_roundtrip`.

Firmware on the simulator (single-core PRU0)

- `test_firmware_self_configures_perif_ch0` — GPCFG/TXCFG/CH0CFG0 written by firmware; continuous mode.
- `test_firmware_prng_and_crc_in_dram` — reads DRAM0 and checks the firmware's PRNG payload and CRC-32 against the reference (isolates codegen from line coding).
- `test_firmware_bert_roundtrip_zero_ber` / `test_firmware_udp_roundtrip_and_valid_ethernet` — end-to-end BER = 0; the UDP capture is checked for a valid IPv4 ethertype and the "Hello World Text" payload.
- `test_pcap_output_roundtrips` — pcap header + record count.
- `test_via_mcp_server` — drives the firmware **through the MCP server wrapper** (`PRUSimulatorMCP.pru_load` / `pru_step` / `pru_memory`) and verifies the frame the firmware built in DRAM0. This satisfies the "test with MCP server" requirement.

Full run and Wireshark trace — `python3 source/pif_eth/driver.py 100` transmits 100 frames of each type (BER = 0 for both) and writes `traces/pif_eth_{bert,udp}.pcap`. Verified externally with `tshark`:

```
$ tshark -r source/pif_eth/traces/pif_eth_udp.pcap -c 1
1    0.000000  192.168.0.1 → 192.168.0.2  UDP 60 1234 → 5678 Len=16
```

with the UDP payload dissecting as `Hello World Text`.

Run the suite:

5. Notable issues resolved during bring-up

- **8-bit AND immediate** — the 10-bit `0x3FF` symbol mask cannot be an immediate; it lives in a register.
- **Stray FIFO push** — `ldi r30, 0` strobes byte 0 of R30, which the perif treats as a TX FIFO push of `0x00`, corrupting the *first* burst only (the second frame was already perfect). Fixed by selecting ch0 with `ldi r30.b2, 0`. This was found by dumping the first burst's sampled bits and seeing exactly 8 leading zero bits before the expected comma.
- **Continuous-mode under-run** — generating payload/CRC mid-stream starves the serialiser; resolved by making each frame its own burst.
- **Timeline advance** — the perif only advances while PRU0 steps, so the firmware spins (not `halt`) and `drain_wait` keeps stepping until the burst fully serialises before the frame counter is published.

6. Token / effort usage

Exact token telemetry is not exposed to the agent, so the figures below are an **approximate** account of where effort went, by phase. The dominant costs were (a) reading the existing simulator/perif source to learn the conventions and (b) the iterative firmware bring-up loop.

Phase	Activity	Relative effort
Exploration	Read perif model (<code>peripheral_interface.py</code> , <code>perif_channel.py</code> ~477 lines, <code>perif_registers.py</code>), MCP server, ISA handlers, existing perif firmware/tests, memory + constant-table config	~30%
Design / clarification	4 clarifying questions; reconciling data flow with the model (single-burst vs. per-frame, timeline advance)	~10%
Python reference layer	codec/crc/prng/frames/pcap/decoder + first test run	~15%
Firmware	~260-line assembly + parse/smoke iterations	~15%
Bring-up / debugging	2 end-to-end debug cycles (stray-push + alignment), 100-frame run	~15%
Docs / tests / report	spec, plan, README, this report, UI helper	~15%

Order-of-magnitude, the session generated ~1,400 lines of code/docs and read several thousand lines of existing source; total token consumption was on the order of a few hundred thousand tokens, weighted toward source reading and the edit/test iterations. The generated code was kept small and reference-driven specifically to keep the debug loop (and thus token spend) short — the firmware was validated against the Python golden model rather than by trial and error.

7. Running with the UI simulator

The UI (`ui/server.py`) keeps a single shared `Simulator`, so the browser and a small seeding client operate on the same DRAM0. The firmware needs the 8b/10b LUT and a control block in DRAM0, which the seeding helper writes over the UI's `write_memory` WebSocket action.

Steps

1. **Start the dashboard** (terminal 1):

```
python ui/server.py
```

Open <http://localhost:8080>.

2. **Load the firmware** in the browser: select core **PRU0**, paste the contents of `source/pif_eth/pif_eth_tx.asm` into the editor, click **Load**. (Load parses the program but does not clear DRAM0, so seed order is flexible — but re-seed after any **Reset**, which clears memory.)

3. **Seed DRAM0 and arm one frame** (terminal 2):

```
python3 source/pif_eth/seed_ui.py bert      # or: udp
```

This writes the LUT (`0x0000`), the control block (`0x0400...`) and sets the go flag (`0x0418 = 1`), arming exactly one frame.

4. **Run**. Click **Run** (there is no step-budget field — Run is a Run/Stop toggle that streams batches of 1,000 instructions on a timer until you click **Stop** or the core halts). A BERT frame is $\approx 20,000$ instructions, so it completes in a fraction of a second; wait a moment, then click **Stop** (the firmware then spins waiting for the next go flag). The I/O window switches to the **Peripheral Interface** view as the firmware writes GPCFG; you can watch channel 0's TX FIFO activity. (For a precise count instead, send a WebSocket

```
{"action": "run", "core": "pru0", "max_steps": 30000}.
```

5. **Observe results** in a **Memory** panel:

- `0x0410` — frame counter (increments to 1),
- `0x0414` — bytes pushed for the burst,
- `0x0500` — the 132-byte (BERT) / 64-byte (UDP) frame the firmware built.

6. **Next frame**: re-run `seed_ui.py` (it re-arms the go flag) and click **Run** again; the counter advances. Repeat as desired.

Headless full run (recommended for the 100-frame BER test + traces):

```
python3 source/pif_eth/driver.py 100
tshark -r source/pif_eth/traces/pif_eth_udp.pcap
```

Note on the UI path: the browser's own **Run** button drives a single core and is ideal for stepping and inspecting one frame at a time. The full 100-frame BER measurement and pcap generation are done by `driver.py`, which also performs the per-frame capture/decode the browser UI does not.